LÖSUNGEN · LEHREREXEMPLAR

W4·2 – Unverändert lassen und die Ladeliste: Lösungen

Zum Arbeitsblatt `AQ_W4_AB2_Unveraendert_und_DynArray.docx` · IDE: `AQ_W4_L4_Hafen_LSG.txt` (Unveraendert, Ladeliste) · alle Zustände, Rückgaben und Laufzeitfehler mit Java (javac 21) und exakten Nachbauten von Stack, Queue und DynArray (Object-basiert, gleiche Exceptions) nachgerechnet; enthaelt, einfuegenSortiert und entferneDuplikate liefern dieselben Ergebnisse wie die Musterlösung · Stapel oben zuerst, Schlange vorne zuerst, Reihung Index 0 zuerst, – = leer

Teil 1 (DS 26)

Aufgabe 1 – Inventur im Lagerhaus (7 BE)

Phase	k	summe	s (oben → unten)	hilf (oben → unten)
Start	–	0	Wein, Salz, Erz, Holz	–
abbauen	Wein	6	Salz, Erz, Holz	Wein
abbauen	Salz	9	Erz, Holz	Salz, Wein
abbauen	Erz	14	Holz	Erz, Salz, Wein
abbauen	Holz	17	–	Holz, Erz, Salz, Wein
zurück	–	17	Holz	Erz, Salz, Wein
zurück	–	17	Erz, Holz	Salz, Wein
zurück	–	17	Salz, Erz, Holz	Wein
zurück	–	17	Wein, Salz, Erz, Holz	–

a) Rückgabe **17**, s am Ende wieder Wein, Salz, Erz, Holz (4 BE). **b)** Rückgabe weiterhin 17, aber s ist **leer** (–): Die Kisten liegen nur noch in der lokalen Variablen hilf und gehen mit dem Ende der Methode verloren (1 BE). **c)** Zurück auf s kommen **neue** Objekte mit gleichen Werten, nicht dieselben Kisten. Alle anderen Referenzen auf die alten Kisten (z. B. aus der Ladeliste) zeigen jetzt auf Kisten, die nicht mehr im Lager liegen; Änderungen an einer Kiste wirken sich nicht mehr auf die andere aus. Inhaltlich gleich ist nicht identisch – der Test vergleicht mit `==` (2 BE).

Aufgabe 2 – Die Umkehr-Maschine (7 BE)

Verfahren	Hilfsstruktur nach dem Umschichten	Ergebnis	umgedreht?
(1) Stapel → Hilfsstapel → Stapel	Hilfsstapel C, B, A (oben zuerst)	s = A, B, C	nein
(2) Stapel → Hilfsschlange → Stapel	Hilfsschlange A, B, C (vorne zuerst)	s = C, B, A	**ja**
(3) Schlange → Hilfsschlange → Schlange	Hilfsschlange A, B, C	q = A, B, C	nein
(4) Schlange → Hilfsstapel → Schlange	Hilfsstapel C, B, A (oben zuerst)	q = C, B, A	**ja**

a) je 1 BE. **b)** Jeder Weg **durch einen Stapel** dreht die Reihenfolge um, ein Weg durch eine Schlange nicht. Bei (1) wird zweimal umgedreht – also wieder die alte Reihenfolge. Kurz: gleiche Strukturart → unverändert, Stapel und Schlange gemischt → umgedreht (1 BE). **c)** Aus dem Hilfsstapel zurück in die Schlange ergibt die **umgekehrte** Reihenfolge (Verfahren 4). Er muss die Elemente ein zweites Mal umdrehen – z. B. erst in einen zweiten Hilfsstapel umschichten und von dort zurück in die Schlange – oder gleich eine Hilfsschlange benutzen (2 BE).

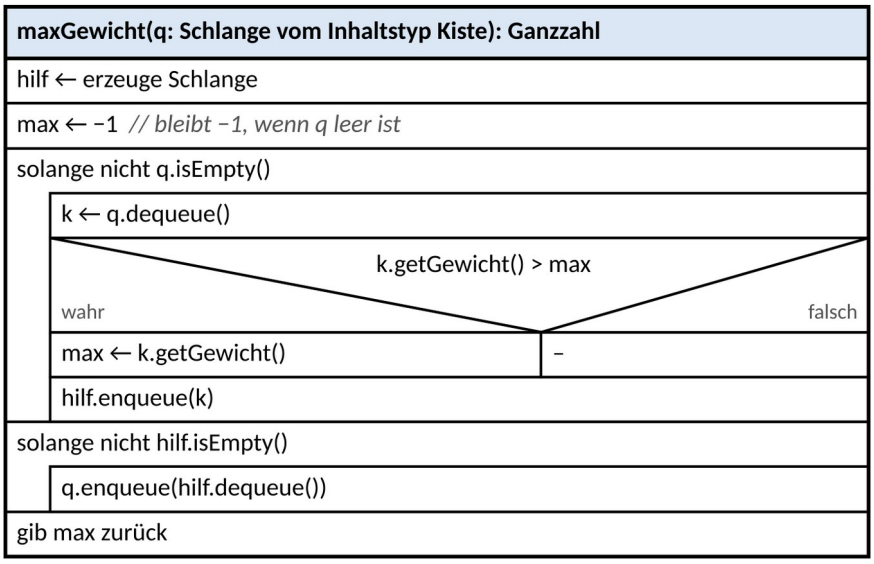
Aufgabe 3 – Papier-Compiler: enthaelt (7 BE)

```
public static boolean enthaelt(Stack s, String name) {
    Stack hilf = new Stack();
    boolean gefunden = false;
    while (!s.isEmpty()) {                // Leer?
        Kiste k = (Kiste) s.pop();        // Cast
        if (k.getName().equals(name)) {   // equals, nicht ==
            gefunden = true;              // merken, NICHT return
        }
        hilf.push(k);                     // dieselbe Kiste
    }
    while (!hilf.isEmpty()) {
        s.push(hilf.pop());               // zurückschichten
    }
    return gefunden;
}
```

a) 5 BE: Hilfsstapel, Schleife mit Leer-Abfrage, Cast, Vergleich mit equals, Ergebnis merken, vollständiges Zurückschichten vor dem return. Identisch mit der Musterlösung von Level 4.3. Mit Java geprüft: Standard-Ladung und "Gold" → true (s danach Wein, Tuch, Salz, Gold, Erz, Holz), "Anker" → false, leerer Stapel → false.

b) Rückgabe true, aber s = **Erz, Holz**: Wein, Tuch und Salz liegen im lokalen Hilfsstapel und sind verloren, Gold wurde entnommen und nicht zurückgelegt (2 BE). Gleichwertige richtige Lösung: vor dem return vollständig zurückschichten (Gold zuerst auf hilf legen).

Aufgabe 4 – Abi-Format: maxGewicht (9 BE)



a) Kernpunkte (4 BE): Kopf mit Signatur · Hilfsschlange · Startwert −1 (liefert zugleich das Ergebnis für die leere Schlange) · Schleife mit Leer-Abfrage, Vergleich und Zwischenlagern **derselben** Kiste · vollständiges Zurückschichten · Rückgabe erst danach. Gleichwertig: Startwert aus der ersten Kiste nach Leer-Abfrage.

Phase	k	max	q (vorne → hinten)	Hilfsschlange
Start	–	-1	Erz, Tuch, Salz, Gold	–
auswerten	Erz 5	5	Tuch, Salz, Gold	Erz
auswerten	Tuch 8	8	Salz, Gold	Erz, Tuch
auswerten	Salz 3	8	Gold	Erz, Tuch, Salz
auswerten	Gold 8	8	–	Erz, Tuch, Salz, Gold
zurück	–	8	Erz	Tuch, Salz, Gold

Phase	k	max	q (vorne → hinten)	Hilfsschlange
zurück	–	8	Erz, Tuch	Salz, Gold
zurück	–	8	Erz, Tuch, Salz	Gold
zurück	–	8	Erz, Tuch, Salz, Gold	–

b) Rückgabe **8**, q danach Erz, Tuch, Salz, Gold (3 BE). Weitere Kontrollwerte: leere Schlange → -1, Standard-Ladung → 8. **c)** n-mal: vorne entnehmen, auswerten, sofort hinten wieder anstellen (Rotieren, Muster 3). Nach n Runden ist jede Kiste genau einmal von vorne nach hinten gewandert – dieselben Kisten stehen wieder in derselben Reihenfolge. Bei n = 0 läuft die Schleife nicht, es kommt kein dequeue auf der leeren Schlange vor (2 BE).

Aufgabe 5 – Level 4.3

Musterlösungen und häufige Fehler: `AQ_W4_L4_Hafen_LSG.txt`, Klasse Unveraendert (10 Tests). Kontrollwerte aus dem Hauptprogramm (Standard-Ladung): 6 Kisten im Lager · Gold im Lager: true · schwerste Kiste Tuch (8 kg, Gleichstand mit Gold, Tuch liegt höher) · unterste Holz (3 kg) · Durchschnitt am Kai 5.5 kg · Lager danach unverändert.

Teil 2 (DS 27)

Aufgabe 6 – Die Ladeliste wächst und schrumpft (8 BE)

Zeile	Anweisung	0	1	2	3	4	5
2	`d.append(18)`	18					
3	`d.append(42)`	18	42				
4	`d.insertAt(1, 7)`	18	7	42			
5	`d.append(42)`	18	7	42	42		
6	`d.insertAt(0, 3)`	3	18	7	42	42	
7	`d.setItem(2, 137)`	3	18	137	42	42	
8	`d.delete(1)`	3	137	42	42		
9	`d.insertAt(d.getLength(), 5)`	3	137	42	42	5	

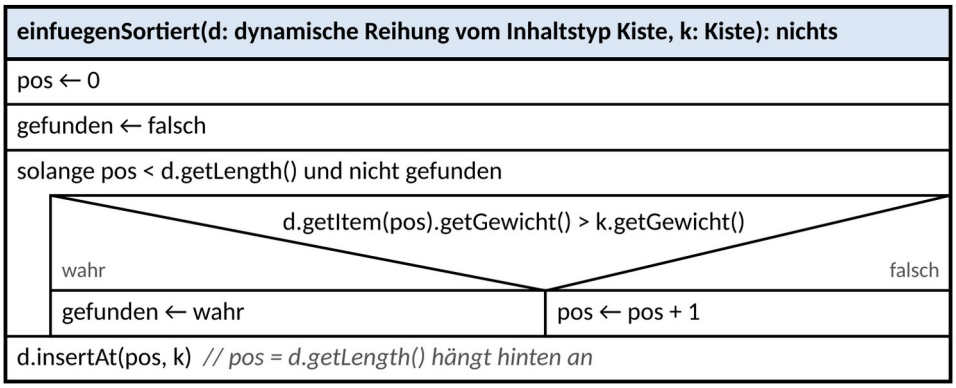
a) 4 BE (je zwei Zeilen 1 BE). Zeile 6 schiebt alle Elemente eine Position nach hinten, Zeile 7 ersetzt nur (Länge bleibt 5), Zeile 8 entfernt 18 und rückt nach vorn, Zeile 9 hängt an. **b)** Ausgabe: **5 42** (1 BE).

```
public static int ersetze(DynArray d, int alt, int neu) {
    int anzahl = 0;
    for (int i = 0; i < d.getLength(); i++) {
        int x = (Integer) d.getItem(i);   // Cast und Unboxing
        if (x == alt) {                   // Zahlen: == ist hier richtig
            d.setItem(i, neu);
            anzahl++;
        }
    }
    return anzahl;
}
```

c) 3 BE. `ersetze(d, 42, 99)` liefert **2**, d = 3, 137, 99, 99, 5. Häufige Fehler: `d.getItem(i) == alt` ohne Cast (Compilerfehler), setItem mit falschem Index, i ≤ getLength().

Aufgabe 7 – Sortiert verladen (9 BE)

a) Nach Blei 6: Salz 3, Erz 5, Wein 6, Blei 6, Tuch 8 (Blei **hinter** Wein) · nach Kork 1: Kork 1, Salz 3, Erz 5, Wein 6, Blei 6, Tuch 8 · nach Anker 9: Kork 1, Salz 3, Erz 5, Wein 6, Blei 6, Tuch 8, Anker 9 (2 BE).



b) Kernpunkte (4 BE): Signatur · Suche der Einfügeposition von vorn · Abbruch bei der **ersten echt schwereren** Kiste (>, nicht ≥ – sonst vor den gleich schweren) · Index-Abfrage pos < d.getLength() **vor** dem Zugriff · insertAt(pos, k) auch für pos = getLength(). Gleichwertig: Schleife mit „und“-Bedingung, wenn der Kurzschluss benannt wird (Index-Abfrage links).

pos	d.getItem(pos)	schwerer als 6 kg?	Aktion
0	Salz 3	nein	pos = 1
1	Erz 5	nein	pos = 2
2	Wein 6	nein	pos = 3
3	Tuch 8	ja	gefunden

c) Danach `d.insertAt(3, Blei)` (2 BE). **d)** Leere Liste: pos < 0 ist sofort falsch, es gibt keinen getItem-Aufruf, insertAt(0, k) ist erlaubt (Ergebnis Kork 1). Schwerste Kiste: Die Schleife endet mit pos = getLength(), bevor getItem(getLength()) aufgerufen wird; insertAt(getLength(), k) hängt an (1 BE).

Aufgabe 8 – Die schwerste Kiste zuerst ausladen (8 BE)

```
public static Kiste entferneSchwerste(DynArray d) {
    if (d.isEmpty()) {                          // Leer? -> kein getItem(0)
        return null;
    }
    int maxIndex = 0;
    for (int i = 1; i < d.getLength(); i++) {
        Kiste k = (Kiste) d.getItem(i);
        Kiste m = (Kiste) d.getItem(maxIndex);
        if (k.getGewicht() > m.getGewicht()) {  // > : die erste gewinnt
            maxIndex = i;
        }
    }
    Kiste schwer = (Kiste) d.getItem(maxIndex); // ERST merken ...
    d.delete(maxIndex);                         // ... DANN loeschen
    return schwer;
}
```

a) 4 BE (Leer-Abfrage, Index des Maximums mit >, Kiste vor dem Löschen sichern, Rückgabe). Spiegelbild von entferneLeichteste aus Level 4.4. **b)** 1. Aufruf: **Gold 8**, d = Holz 3, Erz 5, Salz 3, Tuch 8, Wein 6 · 2.: **Tuch 8**, d = Holz 3, Erz 5, Salz 3, Wein 6 · 3.: **Wein 6**, d = Holz 3, Erz 5, Salz 3 (2 BE).

c) Geliefert wird **Salz 3**: Nach `delete(2)` ist Salz auf Index 2 nachgerückt – zurückgegeben wird der Nachfolger, Gold ist trotzdem verschwunden. Steht die schwerste Kiste **ganz hinten** (z. B. d = Holz 3, Erz 5), gibt es nach dem Löschen keinen Index maxIndex mehr: IndexOutOfBoundsException (mit Java geprüft) (2 BE).

Aufgabe 9 – Die Kiste, die übersprungen wurde (8 BE)

i	d.getLength()	Name an Index i	Aktion	d danach
0	5	Salz	–	Salz, Holz, Holz, Erz, Holz
1	5	Holz	delete(1)	Salz, Holz, Erz, Holz
2	4	Erz	–	Salz, Holz, Erz, Holz
3	4	Holz	delete(3)	Salz, Holz, Erz

a) Nach i = 3 ist i = 4 nicht mehr kleiner als die Länge 3 – die Schleife endet. Ergebnis **Salz, Holz, Erz**: Das zweite Holz (ursprünglich Index 2) wurde übersprungen (3 BE). **b)** Nach `delete(i)` rückt der Nachfolger auf Index i; das anschließende i++ springt über ihn hinweg, er wird nie geprüft. Der Fehler tritt auf, wenn **zwei zu löschende Elemente direkt hintereinander** stehen; stehen sie nie nebeneinander, ist das Ergebnis zufällig richtig – deshalb fällt der Fehler beim Testen leicht durch (2 BE).

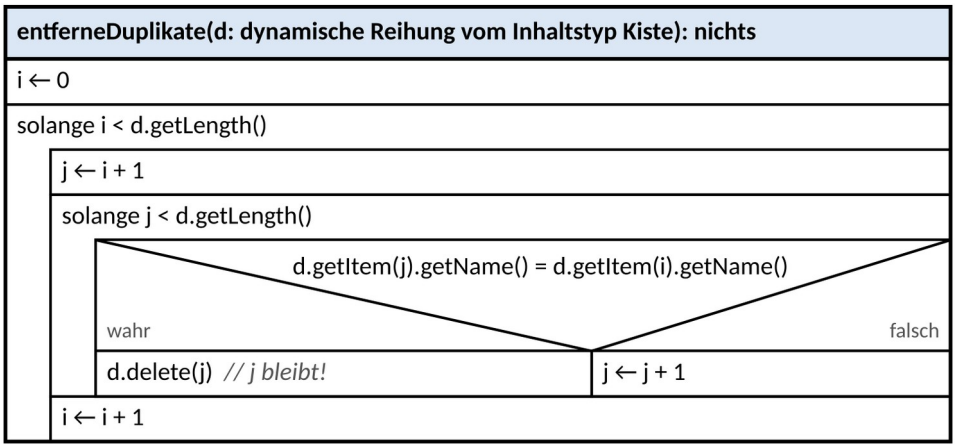
```
// (1) rueckwaerts
for (int i = d.getLength() - 1; i >= 0; i--) {
    Kiste k = (Kiste) d.getItem(i);
    if (k.getName().equals(name)) {
        d.delete(i);
    }
}
```

```
// (2) i nur ohne Loeschen erhoehen
int i = 0;
while (i < d.getLength()) {
    Kiste k = (Kiste) d.getItem(i);
    if (k.getName().equals(name)) {
        d.delete(i);           // i bleibt
    } else {
        i++;
    }
}
```

c) Beide Fassungen liefern Salz, Erz (mit Java geprüft: Salz, Erz bzw. Salz, Erz) (3 BE). Rückwärts funktioniert, weil beim Löschen nur die bereits geprüften Elemente **dahinter** nachrücken.

Aufgabe 10 – Doppelt geladen (7 BE)

a) d = **Holz 3, Erz 5, Salz 3** (1 BE).



b) Kernpunkte (4 BE): äußere Schleife über i, innere ab j = i + 1, Vergleich der Namen (im Struktogramm „=“, in Java equals), nach delete(j) wird j **nicht** erhöht, beide Schleifengrenzen mit der aktuellen Länge. Gleichwertig: innere Schleife rückwärts von getLength() − 1 bis i + 1. Typischer Fehler: j wird nach dem Löschen erhöht → bei drei Holz-Kisten bleibt eine übrig (Java: Holz 3, Erz 5, Holz 2, Salz 3).

c) Ohne Duplikate wird jede Kiste mit jeder späteren verglichen: 5 + 4 + 3 + 2 + 1 = **15** Vergleiche, allgemein **n · (n − 1) / 2** (quadratischer Aufwand). Für die Liste aus a) braucht das Verfahren der Musterlösung nur 7 Vergleiche, weil gelöschte Kisten nicht mehr verglichen werden (2 BE).

Aufgabe 11 – Level 4.4

Musterlösungen und häufige Fehler: `AQ_W4_L4_Hafen_LSG.txt`, Klasse Ladeliste (12 Tests). Kontrollwerte aus dem Hauptprogramm: sortiert eingefügt Holz 3, Salz 3, Erz 5, Wein 6, Gold 8, Tuch 8 ·

Gesamtgewicht 33 kg · Index von Tuch 5 · abspalten(liste, 5): Schwerlast Wein, Gold, Tuch, Rest Holz, Salz, Erz.

Für Schnelle

istAufsteigend: Salz 3, Erz 5, Wein 6 → wahr; Standard-Ladung → falsch; leer → wahr (Java: true false true). **Stapel sortieren:** Ergebnisstapel e; solange s nicht leer: x ← s.pop(); solange e nicht leer und e.top() > x: s.push(e.pop()); e.push(x). Danach liegt der größte Wert oben in e; einmal in einen dritten Stapel umschichten → kleinster oben. Für s = 3, 8, 1, 5 (oben zuerst) ergibt sich 1, 3, 5, 8. **Rotieren um k:** k Runden `q.enqueue(q.dequeue())`; für k > Länge n wirkt es wie k mod n Runden; für die leere Schlange vorher Leer? prüfen (sonst Laufzeitfehler).